

FastAPI Fundamentals

Building a Personal Banking Management System



Routers, Services, and the Repository Pattern



SQLAlchemy ORM Basics



Pydantic for Configuration

Agenda

1. **Introduction to FastAPI:** What and Why?
2. **Project Structure:** The Clean Architecture Approach
3. **Core Components:**
 - **Routers: Handling HTTP Requests**
 - **Services: Implementing Business Logic**
 - **Repositories: Abstracting Data Access**
4. **Database Integration:** SQLAlchemy ORM
5. **Configuration:** Pydantic Settings Management
6. **Conclusion & Key Takeaways**



api_with_best_practices.zip

What is FastAPI?

- A modern, high-performance web framework for building APIs with Python.
- Built on top of **Starlette** (for web parts) and **Pydantic** (for data validation).

Key Features:

 **Fast:** Very high performance, on par with NodeJS and Go.

 **Fast to code:** Increases development speed by 200% to 300%.

 **Fewer bugs:** Reduces about 40% of human-induced errors.

 **Intuitive:** Great editor support, completion everywhere.

 **Easy:** Designed to be easy to use and learn.

 **Automatic Docs:** Interactive API documentation (Swagger UI and ReDoc).

The Project: Personal Banking API

We will build a simple API to manage bank accounts.

Core Features:

- Create a new bank account
- View account details by ID
- Get a list of all accounts
- Deposit money into an account
- Withdraw money from an account

 **Pro Tip:** This project will serve as our practical example for learning FastAPI concepts.

Architecture: Routers, Services, Repositories

This pattern separates concerns, making the application clean, testable, and maintainable.

Router (Controller)

- Receives HTTP requests
- Validates incoming data
 - Calls service methods
- Returns HTTP response

 *A bank teller who receives requests from customers*

Service

- Contains business logic
- Orchestrates operations
- No direct DB interaction

 *The bank manager who approves transactions based on bank rules*

Repository

- Handles DB communication
 - Abstracts data source
- Provides CRUD methods

 *The bank's secure vault and filing system*

Setting Up the Project Structure

A clean structure is key to a scalable application.

```
/banking_api
|-- /app
|   |-- __init__.py
|   |-- main.py           # FastAPI app instance
|   |-- database.py       # Database connection setup
|   |-- config.py         # Configuration settings
|   |-- /routers
|   |   |-- __init__.py
|   |   |-- account_router.py
|   |-- /services
|   |   |-- __init__.py
|   |   |-- account_service.py
|   |-- /repositories
|   |   |-- __init__.py
|   |   |-- account_repository.py
|   |-- /models
|   |   |-- __init__.py
|   |   |-- account.py     # SQLAlchemy model
|   |-- /schemas
|   |   |-- __init__.py
|   |   |-- account_schema.py # Pydantic schemas
|-- requirements.txt
```

The Router Layer

app/routers/account_router.py

- Uses APIRouter to group related endpoints
- Handles request validation using Pydantic schemas
- Depends on the Service layer to perform actions

```
from fastapi import APIRouter, Depends, HTTPException
from ..services.account_service import AccountService
from ..schemas.account_schema import AccountCreate, AccountOut

router = APIRouter(prefix="/accounts", tags=["Accounts"])

@router.post("/", response_model=AccountOut)
def create_account(
    account_data: AccountCreate,
    service: AccountService = Depends()
):
    return service.create_account(account_data)

@router.get("/{account_id}", response_model=AccountOut)
def get_account(account_id: int, service: AccountService = Depends()):
    account = service.get_account(account_id)
    if not account:
        raise HTTPException(status_code=404, detail="Account not found")
    return account
```

 **Pro Tip:** FastAPI's Depends() system automatically creates instances, handles nested dependencies, and makes testing easy by allowing dependency overrides.

Short Exercise 1: Create "Get All Accounts" Endpoint

Task:

Add a new endpoint to `account_router.py` that retrieves a list of all bank accounts.

Requirements:

- **Path:** / (relative to router prefix, so `/accounts/`)
- **HTTP Method:** GET
- **Response:** A list of accounts, `response_model` should be `list[AccountOut]`
- **Logic:** Call a new method in `AccountService` called `get_all_accounts()`

The Service Layer

app/services/account_service.py

- Contains the application's business logic
- Doesn't know about HTTP requests or databases
- Depends on the Repository layer for data

```
from fastapi import Depends
from ..repositories.account_repository import AccountRepository
from ..schemas.account_schema import AccountCreate
from ..models.account import Account

class AccountService:
    def __init__(self, repo: AccountRepository = Depends()):
        self.repo = repo

    def get_account(self, account_id: int):
        return self.repo.get_by_id(account_id)

    def get_all_accounts(self):
        return self.repo.get_all()

    def create_account(self, account_data: AccountCreate) -> Account:
        # Business logic: new accounts start with a balance of 0
        new_account = Account(
            owner_name=account_data.owner_name,
            balance=0.0
        )
        return self.repo.create(new_account)
```

The Repository Layer & SQLAlchemy

app/repositories/account_repository.py

The only layer that interacts directly with the database. Uses SQLAlchemy Session to execute queries.

app/models/account.py (The SQLAlchemy ORM Model)

```
from sqlalchemy import Column, Integer, String, Float
from ..database import Base

class Account(Base):
    __tablename__ = "accounts"

    id = Column(Integer, primary_key=True, index=True)
    owner_name = Column(String, index=True)
    balance = Column(Float, default=0.0)
```

Repository Implementation

app/repositories/account_repository.py

```
from sqlalchemy.orm import Session
from fastapi import Depends
from ..database import get_db
from ..models.account import Account

class AccountRepository:
    def __init__(self, db: Session = Depends(get_db)):
        self.db = db

    def get_by_id(self, account_id: int) -> Account | None:
        return self.db.query(Account).filter(Account.id == account_id).first()

    def get_all(self) -> list[Account]:
        return self.db.query(Account).all()

    def create(self, account: Account) -> Account:
        self.db.add(account)
        self.db.commit()
        self.db.refresh(account)
        return account
```

 **Pro Tip:** The `get_db` dependency is a generator that yields a database session. FastAPI ensures this session is always closed after the request, preventing connection leaks.

Short Exercise 2: Implement Deposit Logic

Task:

Add the logic to deposit money into an account.

Requirements:

- **Repository:** Create `update(account: Account)` method in `AccountRepository`
- **Service:** Create `deposit(account_id: int, amount: float)` method:
 - Fetch the account using repository
 - Return `None` if account doesn't exist
 - Raise `ValueError` if amount is negative
 - Add amount to balance and save
- **Router:** Create POST endpoint at `/accounts/{account_id}/deposit`

Solution 2: Implement Deposit Logic

1. Repository (account_repository.py)

```
# ...
def update(self, account: Account) -> Account:
    self.db.commit()
    self.db.refresh(account)
    return account
```

2. Service (account_service.py)

```
# ...
def deposit(self, account_id: int, amount: float) -> Account | None:
    if amount <= 0:
        raise ValueError("Deposit amount must be positive")

    account = self.repo.get_by_id(account_id)
    if not account:
        return None

    account.balance += amount
    return self.repo.update(account)
```

3. Router (account_router.py)

```
from pydantic import BaseModel

class DepositRequest(BaseModel):
    amount: float

@router.post("/{account_id}/deposit", response_model=AccountOut)
def deposit_money(
    account_id: int,
    request: DepositRequest,
    service: AccountService = Depends()
):
    try:
        updated_account = service.deposit(account_id, request.amount)
        if not updated_account:
            raise HTTPException(status_code=404, detail="Account not found")
        return updated_account
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Configuration Management with Pydantic

Hardcoding database URLs or secret keys is a bad practice. We use Pydantic's BaseSettings to manage configuration.

How it works:

- Reads variables from environment variables or a .env file
- Provides type validation and default values

app/config.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL: str = "sqlite:///./test.db"
    APP_NAME: str = "Personal Banking API"

    class Config:
        env_file = ".env"

settings = Settings()
```

.env file (in the root directory)

```
DATABASE_URL="postgresql://user:password@host:port/dbname"
```

 **Pro Tip:** By creating a single settings object, you have a single source of truth for all configuration. This makes it easy to manage different environments by simply changing environment variables.

Conclusion

Today we learned how to structure a robust FastAPI application using a clean, layered architecture.

 **Routers** handle the web layer

 **Services** contain our core business logic

 **Repositories** abstract away the database

 **SQLAlchemy** provides a powerful ORM for database interaction

 **Pydantic** is essential for both data validation and configuration

This separation of concerns leads to code that is **maintainable**, **scalable**, and **easy to test**.

Key Takeaways

 **Structure Matters:** A good project structure is your best friend. The Router/Service/Repository pattern is a powerful choice.

 **Leverage Dependency Injection:** Use `Depends()` for clean, testable code. It's one of FastAPI's best features.

 **Separate Data Shapes:** Use SQLAlchemy models for database tables and Pydantic schemas for API contracts. Don't mix them.

 **Keep Business Logic in Services:** Your services should be pure Python, unaware of HTTP or SQL. This makes them easy to reuse and test.

 **Externalize Configuration:** Never hardcode settings. Use Pydantic's `BaseSettings` to load config from the environment.



banking_api.zip